

LINUX NETWORKING SERVICES PLAYBOOK

200

Prepared By: Hosni Zaaraoui

Introduction to Part 2: Secure Access and Network Defense

In the world of Linux networking and system administration, three tools stand out as the pillars of secure access and network defense: SSH (Secure Shell), Firewalls, and VPNs (Virtual Private Networks). Together, they ensure secure communication, protect your systems from unauthorized access, and enable safe remote connectivity.

- **SSH** provides a secure way to access and manage remote systems, encrypting all communication to prevent eavesdropping and tampering.
- **Firewalls** act as the first line of defense, controlling incoming and outgoing traffic based on predefined rules to block malicious activity.
- **VPNs** create encrypted tunnels over the internet, allowing remote users to securely access private networks as if they were physically connected.

In this Part 2 of the Linux Networking Services Playbook, we'll take a deep dive into each of these tools. You'll learn how to configure, manage, and troubleshoot SSH, firewalls (using iptables, firewalld, and UFW), and VPNs (with OpenVPN and WireGuard). Whether you're a beginner or an experienced professional, this playbook will equip you with the knowledge to secure your systems and networks effectively.

SSH

1. Introduction to SSH

What is SSH?

SSH is a cryptographic network protocol used for secure communication between two systems. It provides a secure channel over an unsecured network, allowing users to log in to remote systems, execute commands, and transfer files securely.

Why is SSH Important?

- **Security:** Encrypts all communication, protecting against eavesdropping and tampering.
- **Remote Access:** Enables administrators to manage systems from anywhere.
- **File Transfer:** Tools like `scp` and `sftp` allow secure file transfers.
- **Automation:** SSH is essential for scripting and automation in DevOps.

How SSH Works

1. **Connection Initiation:** The client initiates a connection to the SSH server.
2. **Key Exchange:** The client and server exchange cryptographic keys to establish a secure connection.
3. **Authentication:** The client authenticates using a password or public/private key pair.
4. **Encrypted Communication:** All data is encrypted during the session.

2. Setting Up an SSH Server

Step 1: Install SSH Server

On Debian/Ubuntu:



```
sudo apt update
sudo apt install openssh-server
```

On CentOS/RHEL:



```
sudo yum install openssh-server
```

Step 2: Configure the SSH Server

The main configuration file for SSH is located at `/etc/ssh/sshd_config`. Let's explore some key settings:

– Basic Configuration



```
1 # Change the default SSH port (optional)
2 Port 2222
3 # Disable root login for security
4 PermitRootLogin no
5 # Allow specific users to log in
6 AllowUsers alice bob
7 # Enable public key authentication
8 PubkeyAuthentication yes
9 # Disable password authentication (use only key-based auth)
10 PasswordAuthentication no
```

– Explanation of Key Parameters

- **Port:** Changes the default SSH port (22) to reduce brute-force attacks.
- **PermitRootLogin:** Disables direct root login for security.
- **AllowUsers:** Restricts SSH access to specific users.
- **PubkeyAuthentication:** Enables key-based authentication.
- **PasswordAuthentication:** Disables password-based authentication for stronger security.

Step 3: Restart and Enable the SSH Service

Restart the SSH service and enable it to run on boot:



```
# On CentOS/RHEL
sudo systemctl restart sshd
sudo systemctl enable sshd

# On Debian/Ubuntu
sudo systemctl restart ssh
sudo systemctl enable ssh
```

Step 4: Verify the Configuration

Check the status of the SSH service:



```
sudo systemctl status sshd # On CentOS/RHEL

sudo systemctl status ssh  # On Debian/Ubuntu
```

3. SSH Client Configuration

Step 1: Install SSH Client

On **Debian/Ubuntu**:



```
sudo apt update
sudo apt install openssh-client
```

On **CentOS/RHEL**:



```
sudo yum install openssh-clients
```

Step 2: Connect to the SSH Server

Use the **ssh** command to connect to the remote server:



```
ssh username@192.168.1.10 -p 2222 # Replace with your server's IP and port
```

Step 3: Key-Based Authentication

Key-based authentication is more secure than passwords. Here's how to set it up:

1. Generate SSH Key Pair (on the client machine):



```
ssh-keygen -t rsa -b 4096
```

- Save the key pair in the default location (**~/.ssh/id_rsa**).
- Optionally, set a passphrase for added security.

2. Copy the Public Key to the Server:

```
ssh-copy-id -i ~/.ssh/id_rsa.pub username@192.168.1.10 -p 2222
```

3. Test the Connection:

```
ssh username@192.168.1.10 -p 2222
```

4. Advanced SSH Features

– SSH Tunneling

SSH tunneling allows you to securely forward traffic between systems. For example, to forward local port 8080 to a remote server's port 80:

```
ssh -L 8080:localhost:80 username@192.168.1.10 -p 2222
```

– SCP and SFTP

- **SCP (Secure):** files securely between systems.

```
scp -P 2222 file.txt username@192.168.1.10:/path/to/destination
```

- **SFTP (Secure File Transfer Protocol):** Transfer files interactively.

```
sftp -P 2222 username@192.168.1.10
```

– SSH Config File

Simplify SSH connections by creating a config file (`~/.ssh/config`):

```
1 Host myserver
2   HostName 192.168.1.10
3   Port 2222
4   User username
5   IdentityFile ~/.ssh/id_rsa
```

Now, you can connect using:

```
ssh myserver
```

5. Troubleshooting SSH

– Common Issues and Solutions

1. Connection Refused:

- Ensure the SSH service is running on the server.
- Verify the firewall allows traffic on the SSH port.
- Check the SSH server logs:

```
sudo journalctl -u sshd # On CentOS/RHEL
sudo journalctl -u ssh  # On Debian/Ubuntu
```

2. Permission Denied:

- Verify the username and password/key are correct.
- Check the server's `sshd_config` for restrictions (e.g., `AllowUsers`).

3. Slow SSH Connections:

- Disable DNS resolution on the server by adding **UseDNS no** to `/etc/ssh/sshd_config`.

6. Best Practices for SSH

- **Use Key-Based Authentication:** Disable password authentication for stronger security.
- **Change the Default Port:** Reduce the risk of brute-force attacks.
- **Limit User Access:** Restrict SSH access to specific users or groups.
- **Monitor SSH Logs:** Regularly check logs for unauthorized access attempts.
- **Keep SSH Updated:** Ensure the SSH server and client are up-to-date with security patches.

7. Conclusion

SSH is an indispensable tool for secure remote administration in Linux. By mastering its configuration, key-based authentication, and advanced features, you can ensure secure and efficient management of your systems. In the next part of this playbook, we'll explore Firewalls and their role in securing your network.

FIREWALLS

1. Introduction to Firewalls

What is a Firewall?

A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predefined security rules. It acts as a barrier between a trusted internal network and untrusted external networks (like the internet).

Why are Firewalls Important?

- **Access Control:** Restrict unauthorized access to your network.
- **Traffic Filtering:** Block malicious traffic and allow legitimate traffic.
- **Network Segmentation:** Isolate sensitive parts of your network.
- **Compliance:** Meet security requirements for regulatory standards.

Types of Firewalls

1. **Packet-Filtering Firewalls:** Inspect packets and allow or block them based on rules.
2. **Stateful Firewalls:** Track the state of active connections and make decisions based on context.
3. **Application-Layer Firewalls:** Inspect traffic at the application layer (e.g., HTTP, FTP).

2. iptables: The Classic Linux Firewall

iptables is a powerful command-line tool for configuring packet-filtering rules in Linux. It works with netfilter, the Linux kernel's firewall framework.

Step 1: Install iptables

On Debian/Ubuntu:



```
sudo apt update
sudo apt install iptables
```

On CentOS/RHEL:



```
sudo yum install iptables
```

Step 2: Basic iptables Concepts

Tables: iptables uses tables to organize rules. The most common tables are:

- **filter:** Default table for packet filtering.
- **nat:** Used for network address translation (NAT).
- **mangle:** Used for specialized packet alteration.

Chains: Each table contains chains, which are sets of rules. The default chains are:

- **INPUT:** For incoming traffic.
- **OUTPUT:** For outgoing traffic.
- **FORWARD:** For routed traffic.

Rules: Define what to do with packets (e.g., accept, drop, reject).

Step 3: Configure iptables Rules

Here's an example of basic iptables rules for a secure server:

– Allow Loopback Traffic



```
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT
```

– Allow Established Connections



```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j
ACCEPT
```

– Allow SSH (Port 22)



```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

– Allow HTTP (Port 80) and HTTPS (Port 443)



```
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

– Drop All Other Traffic



```
sudo iptables -A INPUT -j DROP
```

– Save iptables Rules

On Debian/Ubuntu:



```
sudo iptables-save > /etc/iptables/rules.v4
```

On CentOS/RHEL:



```
sudo service iptables save
```

Step 4: View and Manage Rules

– List Rules:



```
sudo iptables -L -v -n
```

– Delete a Rule:



```
sudo iptables -D INPUT <rule-number>
```

– Flush All Rules:



```
sudo iptables -F
```

3. firewalld: A Modern Firewall Management Tool

firewalld is a dynamic firewall manager that simplifies firewall configuration. It uses zones and services to manage rules.

Step 1: Install firewalld

On Debian/Ubuntu:



```
sudo apt update  
sudo apt install firewalld
```

On CentOS/RHEL:



```
sudo yum install firewalld
```

Step 2: Start and Enable firewalld



```
sudo systemctl start firewalld  
sudo systemctl enable firewalld
```

Step 3: Configure firewalld

– Zones and Services

- **Zones:** Define the trust level of network connections (e.g., public, internal).
- **Services:** Predefined rules for common applications (e.g., ssh, http, https).

– Allow SSH



```
sudo firewall-cmd --zone=public --add-service=ssh --permanent
```

– Allow HTTP and HTTPS



```
sudo firewall-cmd --zone=public --add-service=http --permanent  
sudo firewall-cmd --zone=public --add-service=https --permanent
```

– Reload firewalld



```
sudo firewall-cmd --reload
```

– View Active Rules



```
sudo firewall-cmd --list-all
```

4. UFW: Uncomplicated Firewall

UFW is a frontend for iptables designed to make firewall management simple and intuitive. It's perfect for users who want an easy-to-use tool without diving into the complexities of iptables.

Step 1: Install UFW

UFW is pre-installed on Ubuntu, but if it's not available, you can install it:

On **Debian/Ubuntu**:



```
sudo apt update
sudo apt install ufw
```

On **CentOS/RHEL** (requires EPEL repository):



```
sudo yum install epel-release
sudo yum install ufw
```

Step 2: Enable and Start UFW

Enable UFW and start it:



```
sudo ufw enable
```

To check the status:



```
sudo ufw status verbose
```

Step 3: Configure UFW Rules

UFW uses simple commands to manage firewall rules. Here are some common configurations:

– Allow SSH

By default, UFW denies all incoming connections. Allow SSH to avoid getting locked out:



```
sudo ufw allow ssh
```

Or specify the port directly:



```
sudo ufw allow 22/tcp
```

– Allow HTTP and HTTPS



```
sudo ufw allow http  
sudo ufw allow https
```

Or by port:



```
sudo ufw allow 80/tcp  
sudo ufw allow 443/tcp
```

– Allow Specific IP Addresses

Allow traffic from a specific IP:



```
sudo ufw allow from 192.168.1.100
```

Allow traffic from a subnet:



```
sudo ufw allow from 192.168.1.0/24
```

– Deny Traffic

Block traffic to a specific port:



```
sudo ufw deny 8080/tcp
```

– Delete a Rule

To delete a rule, first list the rules with numbers:



```
sudo ufw status numbered
```

Then delete the rule by its number:



```
sudo ufw delete <rule-number>
```

Step 4: Advanced UFW Configuration

– Limit Connections

To prevent brute-force attacks, limit connections to a specific port (e.g., SSH):



```
sudo ufw limit ssh
```

– Allow Specific Protocols

Allow UDP traffic (e.g., for DNS):



```
sudo ufw allow 53/udp
```

– Logging

Enable logging to monitor firewall activity:



```
sudo ufw logging on
```

Logs are stored in `/var/log/ufw.log`.

Step 5: Disable or Reset UFW

To disable UFW:



```
sudo ufw disable
```

To reset UFW to its default state (removes all rules):



```
sudo ufw reset
```

UFW Best Practices

- **Always Allow SSH First:** Before enabling UFW, ensure SSH is allowed to avoid locking yourself out.
- **Use Application Profiles:** UFW includes predefined profiles for common services (e.g., ssh, http, https). Use them instead of manually specifying ports.
- **Limit SSH Access:** Use `ufw limit ssh` to protect against brute-force attacks.

- **Regularly Review Rules:** Use `sudo ufw status verbose` to check active rules and remove unnecessary ones.
- **Enable Logging:** Monitor logs for suspicious activity.

5. Comparison: iptables vs. firewalld vs. UFW

Feature	iptables	firewalld	UFW
Ease of Use	Complex	Moderate	Very Easy
Granular Control	Full control	High	Limited
Dynamic Updates	Manual	Yes	Yes
Default on	Most Linux distros	CentOS/RHEL	Ubuntu
Best For	Advanced users	Intermediate users	Beginners

6. Troubleshooting Firewalls

– Common Issues and Solutions

1. Locked Out of the Server:

- Ensure you allow SSH traffic before enabling the firewall.
- Use a console or KVM access to fix rules.

2. Services Not Accessible:

- Verify the correct ports and services are allowed.
- Check for conflicting rules in iptables or firewalld.

3. Firewall Not Persisting After Reboot:

- Ensure rules are saved (`iptables-save` or `firewall-cmd --permanent`).

7. Best Practices for Firewalls

- **Default Deny Policy:** Block all traffic by default and allow only what's necessary.
- **Use Zones in firewalld:** Organize rules based on network trust levels.
- **Regularly Review Rules:** Remove unused or outdated rules.
- **Monitor Logs:** Check firewall logs for suspicious activity.
- **Combine with Other Tools:** Use firewalls alongside IDS/IPS (Intrusion Detection/Prevention Systems) for layered security.

8. Conclusion

Firewalls are a fundamental part of securing your Linux systems and networks. Whether you prefer the granular control of `iptables` or the simplicity of `firewalld`, mastering firewall configuration is essential for protecting your infrastructure. In the next part of this playbook, we'll explore VPNs (Virtual Private Networks) and their role in secure remote access.

VPN

1. Introduction to VPNs

What is a VPN?

A VPN is a technology that creates a secure, encrypted connection over the internet between a user and a private network. It allows remote users to access resources on the private network as if they were directly connected.

Why are VPNs Important?

- **Security:** Encrypts data to protect it from eavesdropping.
- **Privacy:** Masks the user's IP address, enhancing anonymity.
- **Remote Access:** Enables secure access to internal resources from anywhere.
- **Bypass Restrictions:** Allows users to bypass geo-restrictions and censorship.

How VPNs Work

1. **Connection Initiation:** The client connects to the VPN server.
2. **Authentication:** The client authenticates using credentials or certificates.
3. **Encryption:** A secure tunnel is established, encrypting all traffic.
4. **Data Transfer:** The client accesses resources on the private network securely.

2. OpenVPN: A Robust and Flexible VPN Solution

OpenVPN is an open-source VPN solution known for its flexibility and strong security. It uses SSL/TLS for encryption and supports a wide range of configurations.

Step 1: Install OpenVPN

On Debian/Ubuntu:



```
sudo apt update
sudo apt install openvpn easy-rsa
```

On CentOS/RHEL:



```
sudo yum install epel-release
sudo yum install openvpn easy-rsa
```

Step 2: Set Up the Certificate Authority (CA)

OpenVPN uses certificates for authentication. Set up a CA using easy-rsa:

– **the easy-rsa Template:**



```
cp -r /usr/share/easy-rsa/ ~/easy-rsa
cd ~/easy-rsa
```

– Initialize the PKI:



```
./easyrsa init-pki
```

– Build the CA:



```
./easyrsa build-ca
```

– Generate Server Certificate:



```
./easyrsa gen-req server nopass  
./easyrsa sign-req server server
```

– Generate Client Certificate:



```
./easyrsa gen-req client1 nopass  
./easyrsa sign-req client client1
```

– Generate Diffie-Hellman Parameters:



```
./easyrsa gen-dh
```

– Generate HMAC Key:



```
openvpn --genkey --secret ta.key
```

Step 3: Configure OpenVPN Server

– Certificates and Keys:

```
sudo cp ~/easy-rsa/pki/ca.crt /etc/openvpn/server/  
sudo cp ~/easy-rsa/pki/issued/server.crt /etc/openvpn/server/  
sudo cp ~/easy-rsa/pki/private/server.key /etc/openvpn/server/  
sudo cp ~/easy-rsa/pki/dh.pem /etc/openvpn/server/  
sudo cp ~/easy-rsa/ta.key /etc/openvpn/server/
```

– Create Server Configuration:

Edit `/etc/openvpn/server/server.conf`:

```
1 port 1194  
2 proto udp  
3 dev tun  
4 ca ca.crt  
5 cert server.crt  
6 key server.key  
7 dh dh.pem  
8 server 10.8.0.0 255.255.255.0  
9 push "redirect-gateway def1 bypass-dhcp"  
10 push "dhcp-option DNS 8.8.8.8"  
11 keepalive 10 120  
12 tls-auth ta.key 0  
13 cipher AES-256-CBC  
14 user nobody  
15 group nogroup  
16 persist-key  
17 persist-tun  
18 status openvpn-status.log  
19 verb 3
```

– Start and Enable OpenVPN:

```
sudo systemctl start openvpn@server
sudo systemctl enable openvpn@server
```

Step 4: Configure OpenVPN Client

– Client Certificates:

Transfer the following files to the client:

- ca.crt
- client1.crt
- client1.key
- ta.key

– Create Client Configuration:

Create a file named **client.ovpn**:

```
1 client
2 dev tun
3 proto udp
4 remote <server-ip> 1194
5 resolv-retry infinite
6 nobind
7 persist-key
8 persist-tun
9 ca ca.crt
10 cert client1.crt
11 key client1.key
12 tls-auth ta.key 1
13 cipher AES-256-CBC
14 verb 3
```

– Connect to the VPN:

Use the OpenVPN client to connect:



```
sudo openvpn --config client.ovpn
```

3. WireGuard: A Modern and Efficient VPN

WireGuard is a lightweight and high-performance VPN solution that uses state-of-the-art cryptography. It's simpler to configure than OpenVPN and is included in the Linux kernel.

Step 1: Install WireGuard

On Debian/Ubuntu:



```
sudo apt update  
sudo apt install wireguard
```

On CentOS/RHEL:



```
sudo yum install epel-release  
sudo yum install wireguard-tools
```

Step 2: Configure WireGuard Server

– Generate Keys:

```
umask 077  
wg genkey | tee privatekey | wg pubkey > publickey
```

– Create Server Configuration:

Edit `/etc/wireguard/wg0.conf`:

```
1 [Interface]  
2 Address = 10.0.0.1/24  
3 ListenPort = 51820  
4 PrivateKey = <server-private-key>  
5  
6 [Peer]  
7 PublicKey = <client-public-key>  
8 AllowedIPs = 10.0.0.2/32
```

– Start WireGuard:

```
sudo systemctl start wg-quick@wg0  
sudo systemctl enable wg-quick@wg0
```

Step 3: Configure WireGuard Client

– Generate Keys:

```
umask 077  
wg genkey | tee privatekey | wg pubkey > publickey
```

– Create Client Configuration:

Create a file named **wg0.conf**:

```
1 [Interface]  
2 Address = 10.0.0.2/24  
3 PrivateKey = <client-private-key>  
4  
5 [Peer]  
6 PublicKey = <server-public-key>  
7 Endpoint = <server-ip>:51820  
8 AllowedIPs = 0.0.0.0/0
```

– Connect to the VPN:

```
sudo wg-quick up wg0.conf
```

4. Troubleshooting VPNs

– Common Issues and Solutions

1. Connection Issues:

- Verify the server's IP and port are correct.
- Check firewall rules to ensure the VPN port is open.

2. Certificate Errors:

- Ensure the client and server certificates match.
- Regenerate certificates if necessary.

3. Performance Issues:

- Use UDP instead of TCP for better performance.
- Optimize encryption settings.

5. Best Practices for VPNs

- **Use Strong Encryption:** Always use modern encryption algorithms (e.g., AES-256).
- **Regularly Rotate Keys:** Periodically regenerate keys and certificates.
- **Monitor Logs:** Check VPN logs for suspicious activity.
- **Restrict Access:** Use firewall rules to limit VPN access to trusted IPs.

6. Conclusion

VPNs are a critical tool for secure remote access, and both OpenVPN and WireGuard offer robust solutions for Linux users. By mastering their configuration and best practices, you can ensure secure and reliable connectivity for your network.